

I-Villasmil-Ω Formula for Universal Coherence

A mathematical framework to measure coherence in any complex system: physics, economics, AI, biology, neuroscience, organizations, and more.

Quick Start

```
```python
```

```
pip install numpy
```

```
python coherence_calculator.py
```

What is it?

A universal formula where YOU define the variables based on what you're studying.

Formula:

$$C_{\text{total}} = [\sum_{i=1}^6 L_i \cdot (1 - \phi_i) \cdot E_i \cdot f_i] \cdot \Omega_U \cdot R_{\text{fin}} \cdot (1 + 0.037)$$

### Examples

- Physics: Measure quantum coherence
- Economics: Measure market stability
- AI: Measure response quality
- Brain: Measure consciousness integration
- Organizations: Measure team cohesion

## Documentation

- Complete Guide - Everything in one text file
- Python Code - Ready to use
- Examples - Real applications
- Essay - Deep explanation of the theory

## License

MIT License - Use freely

## Citation

```
@misc{villasmil2025coherence,
 title={I-Villasmil-Omega Formula for Universal Coherence},
 author={Villasmil, I.},
 year={2025}
}
```

---

```
ARCHIVO 2: coherence_calculator.py
```

```
```python
```

```
"""
```

I-VILLASMIL- Ω COHERENCE CALCULATOR

Universal coherence measurement for any complex system

Author: I. Villasmil

Version: 1.0.0

License: MIT

"""

import numpy as np

from typing import List, Tuple, Dict, Optional

class CoherenceCalculator:

"""

Calculate coherence using the I-Villasmil-Omega Formula

The formula is:

$$C_{total} = [\sum_{i=1 \rightarrow 6} L_i \cdot (1 - \phi_i) \cdot E_i \cdot f_i] \cdot \Omega_U \cdot R_{fin} \cdot (1 + k)$$

Where variables are defined by YOU based on your system.

"""

def __init__(self, k: float = 0.037, theoretical_max: float = 6.22):

"""

Initialize calculator

Parameters:

k : float

Evolutionary margin constant (default 0.037 = 3.7%)

theoretical_max : float

Theoretical maximum for normalization (default 6.22)

"""

self.k = k

self.theoretical_max = theoretical_max

self.C_star = 0.45 # Observability threshold

def calculate(self,

L: List[float],

phi: List[float],

E: List[float],

f: List[float],

Omega_U: float,

R_fin: float,

layer_names: Optional[List[str]] = None) -> Dict:

"""

Calculate total coherence

Parameters:

L : list of 6 floats

Layer magnitudes [L1, L2, L3, L4, L5, L6]

phi : list of 6 floats

Noise levels [phi1, phi2, phi3, phi4, phi5, phi6]

E : list of 6 floats

Energy/intention [E1, E2, E3, E4, E5, E6]

f : list of 6 floats

Frequencies [f1, f2, f3, f4, f5, f6]

Omega_U : float

Universal context (0 to 1)

R_fin : float

Feedback (0 to 1)

layer_names : list of 6 strings, optional

Names for each layer

Returns:

dict with:

- C_total: raw coherence
- C_normalized: coherence in [0,1]
- contributions: contribution per layer
- interpretation: text interpretation
- is_observable: bool (C >= threshold)

"""

Validate inputs

L = np.array(L)

phi = np.array(phi)

E = np.array(E)

f = np.array(f)

if len(L) != 6 or len(phi) != 6 or len(E) != 6 or len(f) != 6:

raise ValueError("All arrays must have exactly 6 elements (one per layer)")

```
if not (0 <= Omega_U <= 1):
    raise ValueError("Omega_U must be between 0 and 1")

if not (0 <= R_fin <= 1):
    raise ValueError("R_fin must be between 0 and 1")

# Calculate layer contributions
contributions = L * (1 - phi) * E * f

# Sum of layers
layer_sum = np.sum(contributions)

# Apply external factors
C_total = layer_sum * Omega_U * R_fin * (1 + self.k)

# Normalize
C_normalized = C_total / self.theoretical_max

# Determine observability
is_observable = C_normalized >= self.C_star

# Generate interpretation
interpretation = self._interpret(C_normalized, is_observable)

# Build result dictionary
```

```

result = {
    'C_total': C_total,
    'C_normalized': C_normalized,
    'contributions': contributions,
    'layer_sum': layer_sum,
    'Omega_U': Omega_U,
    'R_fin': R_fin,
    'k': self.k,
    'is_observable': is_observable,
    'interpretation': interpretation,
    'layer_names': layer_names if layer_names else [f"Layer {i+1}" for i in range(6)]
}

```

```

return result

```

```

def _interpret(self, C: float, observable: bool) -> str:

```

```

    """Generate text interpretation of coherence value"""

```

```

    if not observable:

```

```

        return f"Below threshold ({self.C_star:.2f}): System is not observable or coherent"

```

```

    elif C < 0.60:

```

```

        return "Basic functional system - minimal coherence"

```

```

    elif C < 0.80:

```

```

        return "Well-organized system - good coherence"

```

```

    elif C < 0.95:

```

```

        return "Highly coherent system - excellent integration"

```

```

    else:

```

```
return "Nearly perfect coherence - exceptional system"
```

```
def print_report(self, result: Dict):
```

```
    """Print detailed report of calculation"""
```

```
    print("=" * 70)
```

```
    print("I-VILLASMIL-Ω COHERENCE CALCULATION REPORT")
```

```
    print("=" * 70)
```

```
    print()
```

```
    print("LAYER CONTRIBUTIONS:")
```

```
    print("-" * 70)
```

```
    for i, (name, contrib) in enumerate(zip(result['layer_names'], result['contributions'])):
```

```
        print(f" {name:30s}: {contrib:.4f}")
```

```
    print("-" * 70)
```

```
    print(f" {'TOTAL (Sum of layers)':30s}: {result['layer_sum']:.4f}")
```

```
    print()
```

```
    print("EXTERNAL FACTORS:")
```

```
    print(f" Omega_U (Universal context): {result['Omega_U']:.3f}")
```

```
    print(f" R_fin (Feedback):      {result['R_fin']:.3f}")
```

```
    print(f" k (Evolutionary margin):  {result['k']:.3f}")
```

```
    print()
```

```
    print("RESULTS:")
```

```
    print(f" C_total (raw):      {result['C_total']:.4f}")
```

```
    print(f" C_normalized [0,1]:   {result['C_normalized']:.4f}")
```

```
print(f" Observable:          {'YES ✓' if result['is_observable'] else 'NO ✗'})  
print()
```

```
print("INTERPRETATION:")  
print(f" {result['interpretation']}")  
print()  
print("=" * 70)
```

```
def compare_systems(self, system1: Dict, system2: Dict,  
                    name1: str = "System 1", name2: str = "System 2"):  
    """Compare coherence between two systems"""  
    C1 = system1['C_normalized']  
    C2 = system2['C_normalized']  
    delta_C = C2 - C1  
    percent_change = (delta_C / C1) * 100 if C1 > 0 else float('inf')  
  
    print("=" * 70)  
    print("SYSTEM COMPARISON")  
    print("=" * 70)  
    print()  
    print(f"{name1:30s}: C = {C1:.4f}")  
    print(f"{name2:30s}: C = {C2:.4f}")  
    print()  
    print(f"Change ( $\Delta C$ ):          {delta_C:+.4f}")  
    print(f"Percent change:          {percent_change:+.1f}%")  
    print()
```

```
if delta_C > 0:
    print(f"→ {name2} is MORE coherent")
elif delta_C < 0:
    print(f"→ {name1} is MORE coherent")
else:
    print(f"→ Systems have EQUAL coherence")

print("=" * 70)
```

```
def example_human_brain():
    """Example: Calculate coherence of awake human brain"""
    print("\n" + "="*70)
    print("EXAMPLE 1: HUMAN BRAIN (AWAKE STATE)")
    print("="*70 + "\n")
```

```
calc = CoherenceCalculator()
```

```
# Define the system
```

```
layer_names = [
    "Thalamus (sensory relay)",
    "Sensory cortex",
    "Associative cortex",
    "Default mode network",
    "Prefrontal cortex",
```

```
"Global integration"
```

```
]
```

```
# Variable values
```

```
L = [0.95, 0.90, 0.85, 0.88, 0.92, 0.80]
```

```
phi = [0.05, 0.08, 0.10, 0.12, 0.08, 0.15]
```

```
E = [0.85, 0.90, 0.92, 0.88, 0.95, 0.85]
```

```
f = [0.90, 0.85, 0.75, 0.60, 0.70, 0.50]
```

```
Omega_U = 0.85 # Coupling with body/environment
```

```
R_fin = 0.75 # Plasticity/homeostasis
```

```
result = calc.calculate(L, phi, E, f, Omega_U, R_fin, layer_names)
```

```
calc.print_report(result)
```

```
def example_ai_system():
```

```
    """Example: Calculate coherence of AI system (Claude)"""
```

```
    print("\n" + "="*70)
```

```
    print("EXAMPLE 2: AI SYSTEM (LANGUAGE MODEL)")
```

```
    print("="*70 + "\n")
```

```
    calc = CoherenceCalculator()
```

```
    layer_names = [
```

```
        "Tokenization",
```

```
"Embeddings",  
"Attention mechanisms",  
"Context memory",  
"Reasoning/inference",  
"Aligned output"  
]
```

```
L = [1.00, 0.95, 0.90, 0.85, 0.80, 0.75]  
phi = [0.02, 0.05, 0.10, 0.15, 0.20, 0.25]  
E = [0.70, 0.90, 0.95, 0.80, 0.85, 0.90]  
f = [0.95, 0.90, 0.85, 0.70, 0.60, 0.75]
```

```
Omega_U = 0.80 # Training knowledge  
R_fin = 0.60 # No inter-session learning
```

```
result = calc.calculate(L, phi, E, f, Omega_U, R_fin, layer_names)  
calc.print_report(result)
```

```
def example_economic_crisis():  
    """Example: Compare economy before and during crisis"""  
    print("\n" + "="*70)  
    print("EXAMPLE 3: ECONOMIC MARKET (PRE-CRISIS VS CRISIS)")  
    print("="*70 + "\n")  
  
    calc = CoherenceCalculator()
```

```
layer_names = [  
    "Individual investors",  
    "Companies",  
    "Industrial sectors",  
    "National market",  
    "Regional markets",  
    "Global economy"  
]
```

```
# Pre-crisis (2006)
```

```
print("Calculating PRE-CRISIS state (2006)...")
```

```
L_pre = [0.85, 0.88, 0.90, 0.85, 0.80, 0.75]
```

```
phi_pre = [0.15, 0.12, 0.10, 0.15, 0.20, 0.25]
```

```
E_pre = [0.80, 0.85, 0.88, 0.82, 0.78, 0.75]
```

```
f_pre = [0.90, 0.80, 0.75, 0.70, 0.60, 0.55]
```

```
result_pre = calc.calculate(L_pre, phi_pre, E_pre, f_pre, 0.85, 0.70, layer_names)
```

```
# During crisis (2008)
```

```
print("\nCalculating CRISIS state (2008)...")
```

```
L_crisis = [0.60, 0.55, 0.50, 0.45, 0.40, 0.35]
```

```
phi_crisis = [0.40, 0.45, 0.50, 0.55, 0.60, 0.65]
```

```
E_crisis = [0.50, 0.45, 0.40, 0.35, 0.30, 0.25]
```

```
f_crisis = [0.95, 0.90, 0.85, 0.80, 0.75, 0.70]
```

```
result_crisis = calc.calculate(L_crisis, phi_crisis, E_crisis, f_crisis, 0.60, 0.30,
layer_names)
```

```
# Compare
```

```
print()
```

```
calc.compare_systems(result_pre, result_crisis, "Pre-crisis 2006", "Crisis 2008")
```

```
def interactive_calculator():
```

```
    """Interactive mode for custom calculations"""
```

```
    print("\n" + "="*70)
```

```
    print("INTERACTIVE COHERENCE CALCULATOR")
```

```
    print("="*70 + "\n")
```

```
    calc = CoherenceCalculator()
```

```
    print("Enter values for your system:")
```

```
    print("(All layer values should be between 0 and 1)\n")
```

```
# Get layer names
```

```
print("First, name your 6 hierarchical layers:")
```

```
layer_names = []
```

```
for i in range(6):
```

```
    name = input(f" Layer {i+1} name: ").strip()
```

```
    layer_names.append(name if name else f"Layer {i+1}")
```

```

print("\nNow enter numerical values:\n")

# Get L values
print("Layer magnitudes (L_i) - how strong is each layer? [0-1]")

L = []

for i, name in enumerate(layer_names):
    while True:
        try:
            val = float(input(f" L_{i+1} ({name}): "))

            if 0 <= val <= 1:
                L.append(val)
                break
            else:
                print(" Must be between 0 and 1")
        except ValueError:
            print(" Please enter a number")

# Get phi values
print("\nNoise levels (phi_i) - how much interference? [0-1]")

phi = []

for i, name in enumerate(layer_names):
    while True:
        try:
            val = float(input(f" phi_{i+1} ({name}): "))

            if 0 <= val <= 1:
                phi.append(val)

```

```

        break

    else:

        print("  Must be between 0 and 1")

except ValueError:

    print("  Please enter a number")


# Get E values

print("\nEnergy/Intention (E_i) - capacity for self-organization [0-1]")

E = []

for i, name in enumerate(layer_names):

    while True:

        try:

            val = float(input(f" E{i+1} ({name}): "))

            if 0 <= val <= 1:

                E.append(val)

                break

            else:

                print("  Must be between 0 and 1")

        except ValueError:

            print("  Please enter a number")


# Get f values

print("\nFrequencies (f_i) - rate of change [0-1, or Hz if you prefer]")

f = []

for i, name in enumerate(layer_names):

    while True:

```

```

try:
    val = float(input(f" f{i+1} ({name}): "))
    if val >= 0:
        f.append(val)
        break
    else:
        print(" Must be >= 0")
except ValueError:
    print(" Please enter a number")

```

Get external factors

```
print("\nExternal factors:")
```

```
while True:
```

```

try:
    Omega_U = float(input(" Omega_U (universal context) [0-1]: "))
    if 0 <= Omega_U <= 1:
        break
    else:
        print(" Must be between 0 and 1")
except ValueError:
    print(" Please enter a number")

```

```
while True:
```

```

try:
    R_fin = float(input(" R_fin (feedback capacity) [0-1]: "))
    if 0 <= R_fin <= 1:

```



```
while True:

    print("\nMENU:")

    print(" 1. Example: Human brain (awake)")

    print(" 2. Example: AI system")

    print(" 3. Example: Economic crisis comparison")

    print(" 4. Interactive calculator (custom system)")

    print(" 5. Exit")


    choice = input("\nSelect option (1-5): ").strip()


    if choice == '1':

        example_human_brain()

    elif choice == '2':

        example_ai_system()

    elif choice == '3':

        example_economic_crisis()

    elif choice == '4':

        interactive_calculator()

    elif choice == '5':

        print("\nThank you for using I-Villasmil-Ω Calculator!")

        break

    else:

        print("Invalid option. Please select 1-5.")


    input("\nPress Enter to continue...")
```

```
if __name__ == "__main__":  
    main()
```